

ASSIGNMENT

Review Data Analysis and Processing

AI6122 Text Data Management & Processing

September 2026

NANYANG TECHNOLOGICAL UNIVERSITY

1 Objective

The objective of this assignment is to familiarize you with the main components of end-to-end text management, processing, and search applications, as well as the challenges faced by each component and their corresponding solutions. Through this assignment, you will also gain hands-on experience with various packages available for information retrieval and natural language processing tasks.

2 Assignment Format

1. This is a group assignment.
2. One report is to be submitted by *each group*, and all members of the same group receive the same grade. However, the **contributions of individual members** to the assignment shall be *clearly indicated* in the report. Group size is not a factor in grading.
3. You may use ANY programming language of your choice, *e.g.,* Java, Python, or C#.
4. You may use any NLP, IR, or machine learning library or software, as long as its license allows free use for educational and/or research purposes.
5. You may use GenAI tools while developing this project. However, you are expected to understand any code you submit and to defend all of your assumptions and design decisions.

3 Assignment (100 marks)

The assignment consists of the following components: Dataset Analysis (20 marks), Development of a Search Engine (40 marks), Trend Analysis (30 marks), and Application (10 marks).

3.1 Dataset

We will use the Yelp Open Dataset (`https://www.yelp.com/dataset`). You should read the dataset documentation before working on the assignment. In this assignment, we will use two files from the dataset: business.json and review.json.[1] There is no need to download the full dataset.

3.2 Dataset Analysis – Empirics & Custom Tokenization (20 Marks)

Before building complex systems, you must understand the statistical properties of your text data and how preprocessing affects them.

3.2.1 Validating Zipf's Law & Heaps' Law

- **Zipf's Law:** Compute the term frequencies across your dataset sample. Plot the log-log graph of term frequency versus term rank to empirically validate Zipf's Law.

[1] Due to its large size, one dataset download per group is strongly encouraged.

- **Heaps' Law:** Using different sample sizes of your dataset (*e.g.,* 10%, 30%, 50%, and 100%), estimate the parameters k and β of Heaps' Law ($V = kN^{\beta}$) to model and predict vocabulary growth. Discuss your findings in the report.

3.2.2 The Custom Tokenizer Challenge

Standard tokenizers may fail on noisy web text. In this task, you will develop a tokenizer specifically for the dataset.

- Write a custom, regex-based tokenizer from scratch to handle noisy Yelp reviews (*e.g.,* preserving emojis, handling contractions like "don't", and managing run-on words like "the-best-food").
- Compare your custom tokenizer's vocabulary size and qualitative output against those of standard tokenizers (*e.g.,* NLTK's whitespace or WordPunct tokenizers).

3.3 Search Engine – Inside the Ranking Black Box (40 Marks)

Instead of treating search engines as black boxes, you will construct the indexing pipeline and implement your own ranking formulas from scratch.

3.3.1 Indexing

Construct an inverted index for the review dataset. You may use a standard library such as Lucene to handle storage and Boolean retrieval, or build a simple Python dictionary-based inverted index.

3.3.2 Ranking Implementation

Implement the following scoring functions **from scratch** (do not use built-in ranking modules):

- **Vector Space Model:** Implement TF-IDF scoring with cosine similarity.
- **Okapi BM25:** Implement the BM25 formula. You must correctly handle the parameters k_1 and b, and compute the average document length globally during the indexing phase.

Analysis: Run a set of at least five diverse queries on both implementations. You may also run the same queries and compare your results with those returned by a standard library such as Lucene, e.g., the built-in ranking modules.

3.4 Trend Analysis – Lexicons & Temporal Associations (30 Marks)

Analyze how sentiments and semantic associations change over time without relying on deep learning models.

3.4.1 Lexicon-Based Sentiment Mining

- Use classic lexicons (*e.g.,* VADER or SentiWordNet) to score the sentiment of reviews.
- Track and visualize how the sentiment of a specific business sector (*e.g.,* "Italian Restaurants" vs. "Fast Food") has shifted across different years or seasons in the dataset. Note the limitations of lexicon-based methods (*e.g.,* handling sarcasm) in your report.

3.4.2 Term Co-occurrence Drift

- Select a target aspect term (*e.g.,* "service", "price", or "atmosphere").
- Use classic association metrics such as Pointwise Mutual Information (PMI) to extract the top N words that co-occur most strongly with your target term in different years.
- Analyze how the semantic associations have drifted over time (*e.g.,* did the words associated with "service" change from year to year?).

3.5 Application (10 marks)

Define and develop a simple NLP/IR application based on the dataset. An example application is detecting sentences containing a *comparison* in reviews. In a sentence containing a "comparison", the reviewer compares the business being reviewed with other businesses that they have visited. Note that, here, an application means a small tool (or piece of code) to analyze or mine the data. It does not mean a web-based application or mobile app. A command-line interface is sufficient; a GUI or web-based interface will not contribute to grading.

4 Submission of Report and Source Code

4.1 *Final Report in Hardcopy*

- The hardcopy report must be submitted on or before **5 Nov 2026** (Thursday), during our evening class.
- The report must use the provided cover page, and the main content shall be formatted according to the ACM "sigconf" proceedings template[2] (either MS Word or LaTeX). The main content of the report ***must not exceed 10 pages***, *i.e.,* excluding the cover page, references, and appendix.
- DO NOT include all of the source code and complete result sets in your report. However, you must include important *code snippets* for the main functions of your tasks. You should cite all third-party libraries used in your assignment.
- The report shall be printed double-sided whenever possible. A plastic cover or ring binding incurs a 2% penalty.
- Before submission, please read the hardcopy of your report. **Make sure all words and pictures in your report are readable**.

4.2 *Final Report in Softcopy, Source Code, and Documentation*

- Report_Gxx.PDF shall be submitted through NTULearn using the link for *report submission*. This PDF file shall have the same content as the submitted hardcopy report. Note: xx is your group number. Only one member from each group needs to submit the softcopy on behalf of the group.

[2] `https://www.acm.org/publications/proceedings-template`

- An Src_Gxx.zip file containing Readme.txt and the SourceCode folder shall be submitted through NTULearn using the link for *source code submission*. Only one member from each group needs to submit the softcopy on behalf of the group.
  - Readme.txt shall include:
    - Links to download any third-party libraries used in your assignment.
    - An installation guide on how to set up your system, and how to use it (*e.g.,* command lines, input format, parameters).
    - Explanations of sample output obtained from your system.
  - The SourceCode folder shall contain **only your own source code**. The dataset and the libraries shall **NOT** be included in the softcopy submission, in order to minimize the file size.
- Softcopy submission deadline: ***5 Nov 2026, 11:59 PM***. Late submissions are allowed but will be penalized by 5% for every calendar day (until the mark reaches zero). The softcopy can be submitted at most three times; only the last submission will be graded and time-stamped.